

CHAPTER – 1

INTRODUCTION

1.1 STOCK PRICE PREDICTION

Stock Market prediction and analysis is the act of trying to determine the future value of a company stock or other financial instrument traded on an exchange. Stock market is the important part of economy of the country and plays a vital role in the growth of the industry and commerce of the country that eventually affects the economy of the country. Both investors and industry are involved in stock market and wants to know whether some stock will rise or fall over certain period of time. The stock market is the primary source for any company to raise funds for business expansions. It is based on the concept of demand and supply. If the demand for a company's stock is higher, then the company share price increases and if the demand for company's stock is low then the company share price decrease. Due to involvement of many number of industries and companies, it contain very large sets of data from which it is difficult to extract information and analyse their trend of work manually. Stock market analysis and prediction will reveal the market patterns and predict the time to purchase stock. The successful prediction of a stock's future price could yield significant profit. This is done using large historic market data to represent varying conditions and confirming that the time series patterns have statistically significant predictive power for high probability of profitable trades and high profitable returns for the competitive business investment.

Machine learning is a method of data analysis that automates analytical model building. Using algorithms that iteratively learn from data, machine learning allows computers to find hidden insights without being explicitly programmed where to look. Application of Machine learning algorithms is generally focused on technical analysis but incorporation of the concepts of fundamental analysis into machine learning can be beneficial. This project tells about how various efforts have been taken in the application of Machine Learning to Stock forecasting and also suggests new potent ideas that can be worked upon. Recently, a lot of interesting work has been done in the area of applying Machine Learning Algorithms for analysing price patterns and predicting stock prices and index changes.

1.2 RECURRENT NEURAL NETWORKS (RNN)

A recurrent neural network (RNN) is a type of artificial neural network which uses sequential data or time series data. These deep learning algorithms are commonly used for ordinal or temporal problems, such as language translation, natural language processing (nlp), speech recognition, and image captioning; they are incorporated into popular applications such as Siri, voice search, and Google Translate. Like feedforward and convolutional neural networks (CNNs), recurrent neural networks utilize training data to learn. They are distinguished by their “memory” as they take information from prior inputs to influence the current input and output. While traditional deep neural networks assume that inputs and outputs are independent of each other, the output of recurrent neural networks depend on the prior elements within the sequence. While future events would also be helpful in determining the output of a given sequence, unidirectional recurrent neural networks cannot account for these events in their predictions.

Another distinguishing characteristic of recurrent networks is that they share parameters across each layer of the network. While feedforward networks have different weights across each node, recurrent neural networks share the same weight parameter within each layer of the network. That said, these weights are still adjusted in the through the processes of backpropagation and gradient descent to facilitate reinforcement learning.

Recurrent neural networks leverage backpropagation through time (BPTT) algorithm to determine the gradients, which is slightly different from traditional backpropagation as it is specific to sequence data. The principles of BPTT are the same as traditional backpropagation, where the model trains itself by calculating errors from its output layer to its input layer. These calculations allow us to adjust and fit the parameters of the model appropriately.

1.3 LONG SHORT TERM MEMORY (LSTM)

To solve the problem of Vanishing and Exploding Gradients in a Deep Recurrent Neural Network, many variations were developed. One of the most famous of them is the Long Short Term Memory Network(LSTM). In concept, an LSTM recurrent unit tries to “remember” all the past knowledge that the network is seen so far and to “forget” irrelevant data. This is done by introducing different activation function layers called “gates” for different purposes. Each LSTM recurrent unit also maintains a vector called the Internal Cell State which conceptually describes the information that was chosen to be retained by the previous LSTM recurrent unit.

LSTM networks are the most commonly used variation of Recurrent Neural Networks (RNNs). The critical component of the LSTM is the memory cell and the gates (including the forget gate but also the input gate), inner contents of the memory cell are modulated by the input gates and forget gates. Assuming that both of the segue he are closed, the contents of the memory cell will remain unmodified between one time-step and the next gradients gating structure allows information to be retained across many time-steps, and consequently also allows group that to flow across many time-steps. This allows the LSTM model to overcome the vanishing gradient properly occurs with most Recurrent Neural Network models. The basic workflow of a Long Short Term Memory Network is similar to the workflow of a Recurrent Neural Network with the only difference being that the Internal Cell State is also passed forward along with the Hidden State.

Long Short Term Memory Network consists of four different gates for different purposes as described below:-

- **Forget Gate(f)** : At forget gate the input is combined with the previous output to generate a fraction between 0 and 1, that determines how much of the previous state need to be preserved (or in other words, how much of the state should be forgotten). This output is then multiplied with the previous state
- **Input Gate(i)** : Input gate operates on the same signals as the forget gate, but here the objective is to decide which new information is going to enter the state of LSTM. The output of the input gate (again a fraction between 0 and 1) is multiplied with the output

of tan h block that produces the new values that must be added to previous state. This gated vector is then added to previous state to generate current state

- **Output Gate(o)** : At output gate, the input and previous state are gated as before to generate another scaling fraction that is combined with the output of tanh block that brings the current state. This output is then given out. The output and state are fed back into the LSTM block.

1.4 BIDIRECTIONAL - LONG SHORT TERM MEMORY (BI-LSTM)

Bidirectional recurrent neural networks (BRNN) connect two hidden layers of opposite directions to the same output. With this form of generative deep learning, the output layer can get information from past (backwards) and future (forward) states simultaneously. Invented in 1997 by Schuster and Paliwal, BRNNs were introduced to increase the amount of input information available to the network. For example, multilayer perceptron (MLPs) and time delay neural network (TDNNs) have limitations on the input data flexibility, as they require their input data to be fixed. Standard recurrent neural network (RNNs) also have restrictions as the future input information cannot be reached from the current state. On the contrary, BRNNs do not require their input data to be fixed. Moreover, their future input information is reachable from the current state.

BRNN are especially useful when the context of the input is needed. For example, in handwriting recognition, the performance can be enhanced by knowledge of the letters located before and after the current letter. The principle of BRNN is to split the neurons of a regular RNN into two directions, one for positive time direction (forward states), and another for negative time direction (backward states). Those two states' output are not connected to inputs of the opposite direction states. The general structure of RNN and BRNN can be depicted in the right diagram. By using two time directions, input information from the past and future of the current time frame can be used unlike standard RNN which requires the delays for including future information.

1.5 FBPROPHET

Forecasting at scale - Prophet is a forecasting procedure implemented in R and Python. It is fast and provides completely automated forecasts that can be tuned by hand by data scientists and analysts. Prophet is a procedure for forecasting time series data based on an additive model where non-linear trends are fit with yearly, weekly, and daily seasonality, plus holiday effects. It works best with time series that have strong seasonal effects and several seasons of historical data. Prophet is robust to missing data and shifts in the trend, and typically handles outliers well. Prophet is open source software released by Facebook's Core Data Science team.

FEATURES :

1. Accurate and fast -

Prophet is used in many applications across Facebook for producing reliable forecasts for planning and goal setting. We've found it to perform better than any other approach in the majority of cases. We fit models in Stan so that you get forecasts in just a few seconds.

2. Fully automatic -

Get a reasonable forecast on messy data with no manual effort. Prophet is robust to outliers, missing data, and dramatic changes in your time series.

3. Tunable forecasts -

The Prophet procedure includes many possibilities for users to tweak and adjust forecasts. You can use human-interpretable parameters to improve your forecast by adding your domain knowledge.

4. Available in R or Python -

We've implemented the Prophet procedure in R and Python, but they share the same underlying Stan code for fitting. Use whatever language you're comfortable with to get forecasts.

CHAPTER – 2

LITERATURE REVIEW

2.1 NOVEL OPTIMIZATION APPROACH FOR STOCK PRICE FORECASTING USING MULTI-LAYERED SEQUENTIAL LSTM

Abdul Quadir Md, Sanjit Kapoor, Chris Junni A.V, Arun Kumar Sivaraman, Kong Fah Tee, Sabireen H, Janakiraman N has proposed the new approach for stock price prediction based on a Multi-Layer Sequential Long Short Term Memory (MLS LSTM) model that makes use of the Adam optimizer. Various algorithms like LSTM, Multi-layered LSTM along with various performance metrics like Normalised Root Mean Squared Error (RMSE), Mean Absolute Percentage Error measure (MAPE) and R^2 measure to analyse the efficiency of the trained models. The model is trained with stock data from the year 2017 till the year 2020. The dataset used is very less compared to the accuracy metrics obtained and MLS LSTM achieved a prediction accuracy of 95.9% and 98.1% on the testing data set. This MLS LSTM model is sensitive to overfitting and we must train the model with more data.[1]

2.2 MULTI-SCALE LOCAL CUES AND HIERARCHICAL ATTENTION-BASED LSTM FOR STOCK PRICE TREND PREDICTION

Xiao Teng, Xiang Zhang, Zhigang Luo has proposed a novel multi-scale local cues and hierarchical attention-based LSTM (MLCA-LSTM) model to predict stock price trends by applying local descriptors. This model integrates local descriptors, multi-scale sliding windows, multi-branch LSTM modules and hierarchical attention mechanism altogether to boost stock price trend prediction performance. The model uses the approach of sliding window to predict the stock prices for the next forecasting days.

In contrast with LSTM-based models, MLCA-LSTM degenerates more slowly than them. The reason is probably that the use of local descriptors reduces the length of original time series, which makes MLCA-LSTM capable of dealing with longer dependency relationship. The paper also gives conclusion that the number of the hidden states has a tremendous influence on the performance of both MLCA-LSTM and Adv-LSTM. MLCA-LSTM and Adv-LSTM tend to achieve better performance when the number of hidden states is set to 20 or 25. However, MLCA-LSTM yields better prediction performance than Adv-LSTM in most cases, and achieves the best result when the number of hidden states is set to 20.[2]

2.3 REAL-TIME FORECASTING OF TIME SERIES IN FINANCIAL MARKETS USING SEQUENTIALLY TRAINED DUAL-LSTMS

Kelum Gajamannage, Yonggi Park, Dilhani I. Jayathilake has proposed that classic linear models have no capability of capturing non-linear and non-stationary relationships in the data, nonlinear models might not cope with certain nonlinear and non-stationary signals as the underlying model does not fit with the data. The model is a many-to-one dual-LSTM with sequential data sampled using a moving window approach such that the succeeding window is shifted forward by one data instance from the preceding window. Sequential window training maximizes the performance of this dual-LSTM as it accelerates LSTMs' learning capability as well as it increases the LSTMs' robustness to new data. The performance analysis covers the dual-LSTM applied to nine time series obtained from three financial markets, stocks (Apple, Microsoft, Google), cryptocurrencies (Bitcoin, Ethereum, Cardano), and commodities (gold, crude oil, natural gas).[3]

2.4 FORECASTING TURNING POINTS IN STOCK PRICE BY APPLYING A NOVEL HYBRID CNN-LSTM-RESNET MODEL FED BY 2D SEGMENTED IMAGES

Pouya Khodaei, Akbar Esfahanipour, Hassan Mehtari Taheri have proposed a CNN-LSTM-RESNET model which presented an innovative 3D-CNN for action detection, in which the features were extracted from the temporal and spatial dimensions. Piecewise Linear Regression (PLR) is a method to split the time series of close price into various segments using a predetermined Threshold. Image Segmentation is used to determine the objects and boundaries in a specific image. For the sake of clarity, a fitness function was used to obtain the Threshold of each stock's close price time series by considering its fluctuation. This model also developed a balancing algorithm by considering an over-sampling method, which considers the neighbors of a stock date. However, one of the drawbacks of their approach was creating unbalanced classes in which the number of holding days was far in excess of buying and selling points.

CHAPTER – 3

SYSTEM ANALYSIS

3.1 PROBLEM STATEMENT

The rate of investment and business opportunities in the Stock market can increase if an efficient algorithm could be devised to predict the short term price of an individual stock. Stock price forecasting is among one of the most challenging problems that financial institutions, businesses, and individual investors face. Many factors impact the validity of stock price forecasts, including economics, political contexts, and investor psychology. A company with healthy finances will perform better in the stock market than a company with less-than-ideal finances. The usage of machine learning paradigms in the financial sector, especially the stock market, has been on the rise for a long time. Technical analysis is mainly based on a study of past and present stock prices to predict the future movements of stock prices. Stock market prediction is not linear, thus making it harder to predict a corporation's stock prices in a specific market. Our main objective is to forecast and predict the stock prices in order to make more informed and accurate investment decisions.

3.2 EXISTING SYSTEM

The existing systems available for stock price prediction and forecasting are trained with different models involving Machine Learning and Deep Learning. Machine Learning Algorithms like Random Forest and Support Vector Machines are used which produce lesser accuracy. Deep Learning Algorithms like LSTM, ANN, RNN models trained are prone to overfitting issues. The available models are also trained with static data using csv files which are downloaded and trained. This limits the dataset size available. Loss and Accuracy parameters also varies with different models and visualisation is not proper, precise and consistent. Some papers focus only on the limited attributes rather than all the important attributes needed for financial analysis.

3.2.1 LIMITATIONS OF EXISTING SYSTEM

- Static datasets
- Selection Bias
- Incorrect Pre-processing
- Overfitting issues
- Lesser Accuracy
- Higher Loss
- Some of the evidences are survey-based.

3.3 PROPOSED SYSTEM

Our model is developed for the stock price prediction to help investors in making financial decisions. In most researches it focuses on "lowest price buy", "highest selling price". On the "lowest buy" and "highest selling" strategy of stocks occurs when stocks are at the lowest price and sell shares when prices are highest. For stock price predictions Bidirectional - LSTM technique is used with back propagation the dataset was pre-processed and tuned up for real analysis. Hence, we can upload stock price history i.e. open price, highest price, lowest price and close price of the day. Secondly, after pre-processing the data, our model analyses stock price history and gives input to the Back propagation algorithm. In addition, the proposed model also examines the use of the prediction system in real-world settings and issues associated with the accuracy of the overall values given. The back propagation gives output as final predicted rate comes. The proposed system can get the output graph of stock price prediction. The successful prediction of the stock will be a great asset for the stock market institutions and will provide real-life solutions to the problems that stock investors face. The backward propagation of errors, of back propagation, is a common method of training artificial neural networks.

3.3.1 ADVANTAGE OF THE PROPOSED SYSTEM

- Improved Decision Making
- Sliding Window Approach
- Better Visualisations
- Better Idea about Entry and Exit Points
- Increased Accuracy
- Reduced Bias
- Less time complexity.
- Less execution time.

3.4 REQUIREMENT ANALYSIS AND SPECIFICATION

3.4.1 REQUIREMENT DESCRIPTION

FRONT END : JUPYTER NOTEBOOK

The Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations and narrative text. Uses include: data cleaning and transformation, numerical simulation, statistical modelling, data visualization, machine learning, and much more. The software requirement specification is created at the end of the analysis task. The function and performance allocated to software as part of system engineering are developed by establishing a complete information report as functional representation, a representation of system behaviour, an indication of performance requirements and design constraints, appropriate validation criteria.

FEATURES OF JUPYTER NOTEBOOK

- In-browser editing for code, with automatic syntax highlighting, indentation, and tab completion/introspection.
- The ability to execute code from the browser, with the results of computations attached to the code which generated them.
- Displaying the result of computation using rich media representations, such as HTML, LaTeX, PNG, SVG, etc.
- For example, publication-quality figures rendered by the matplotlib library, can be included inline.
- In-browser editing for rich text using the Markdown markup language, which can provide commentary for the code, is not limited to plain text.
- The ability to easily include mathematical notation within markdown cells using LaTeX, and rendered natively by MathJax.

NOTEBOOK DOCUMENTS

Notebook documents contain the inputs and outputs of an interactive session as well as additional text that accompanies the code but is not meant for execution. In this way, notebook files can serve as a complete computational record of a session. interleaving executable code with explanatory text, mathematics, and rich representations of resulting objects. These documents are internally JSON files and are saved with the ipynb extension. Since JSON is a plain text format, they can be version-controlled and shared with colleagues. Notebooks may be exported to a range of static formats, including HTML (for example, for blog posts), reStructuredText, LaTeX, PDF, and slide shows, via the nbconvert command.

Furthermore, any ipynb notebook document available from a public URL can be shared via the Jupyter Notebook Viewer (nbviewer). This service loads the notebook document from the URL and renders it as a static web page. The results may thus be shared with a colleague, or as a public blog post, without other users needing to install the Jupyter notebook themselves.

FEATURES OF PYTHON

Python is a high-level general-purpose programming language whose design philosophy mostly emphasizes code readability with the use of significant code. Writing code in Python for both small and large scale projects becomes very easy. Python has a variety of features to offer to its users.

1. EASY TO CODE

Python is a very high-level programming language, yet it is effortless to learn. Anyone can learn to code in Python in just a few hours or a few days. Mastering Python and all its advanced concepts, packages and modules might take some more time. However, learning the basic Python syntax is very easy, as compared to other popular languages like C, C++, and Java.

2. EASY TO READ

Python code looks like simple English words. There is no use of semicolons or brackets, and the indentations define the code block. It can tell what the code is supposed to do simply by looking at it.

3. FREE AND OPEN-SOURCE

Python is developed under an OSI-approved open source license. Hence, it is completely free to use, even for commercial purposes. It doesn't cost anything to download Python or to include it in your application. It can also be freely modified and re-distributed. Python can be downloaded from the official Python website.

4. ROBUST STANDARD LIBRARY

Python has an extensive standard library available for anyone to use. This means that programmers don't have to write their code for every single thing unlike other programming languages.

There are libraries for image manipulation, databases, unit-testing, expressions and a lot of other functionalities. In addition to the standard library, there is also a growing collection of thousands of components, which are all available in the Python Package Index.

5. INTERPRETED

When a programming language is interpreted, it means that the source code is executed line by line, and not all at once. Programming languages such as C++ or Java are not interpreted, and hence need to be compiled first to run them. There is no need to compile Python because it is processed at runtime by the interpreter.

6. PORTABLE

Python is portable in the sense that the same code can be used on different machines. Suppose you write a Python code on a Mac. Suppose you want to run it on Windows or Linux later, no need to make any changes in it. As such, there is no need to write a program multiple times for several platforms.

7. OBJECT-ORIENTED AND PROCEDURE-ORIENTED

A programming language is object-oriented if it focuses design around data and objects, rather than functions and logic. On the contrary, a programming language is procedure-oriented if it focuses more on functions (code that can be reused). One of the critical Python features is that it supports both object-oriented and procedure-oriented programming.

8. EXTENSIBLE

A programming language is said to be extensible if it can be extended to other languages. Python code can also be written in other languages like C++, making it a highly extensible language.

9. EXPRESSIVE

Python needs to use only a few lines of code to perform complex tasks. For example, to display Hello World, simply need to type one line - `print("Hello World")`. Other languages like Java or C would take up multiple lines to execute this.

10. SUPPORT FOR GUI

One of the key aspects of any programming language is support for GUI or Graphical User Interface. A user can easily interact with the software using a GUI. Python offers various toolkits, such as Tkinter, wxPython and JPython, which allows for GUI's easy and fast development.

11. DYNAMICALLY TYPED

Many programming languages need to declare the type of the variable before runtime. With Python, the type of the variable can be decided during runtime. This makes Python a dynamically typed language.

For example, if you have to assign an integer value 20 to a variable "x", you don't need to write `int x = 20`. You just have to write `x = 15`.

12. HIGH-LEVEL LANGUAGE

Python is a high-level programming language because programmers don't need to remember the system architecture, nor do they have to manage the memory. This makes it super programmer-friendly and is one of the key features of Python.

13. SIMPLIFY COMPLEX SOFTWARE DEVELOPMENT

Python can be used to develop both desktop and web apps and complex scientific and numerical applications. Python's data analysis features help you create custom big data solutions without so much time and effort.

You can also use the Python data visualization libraries and APIs to present data in a more appealing way. Several advanced software developers use Python to accomplish high-end AI and natural language processing tasks.

14. OTHER ADVANCED PROGRAMMING FEATURES

Python contains several advanced programming features such as generators (used to create iterators with a different approach than most other languages) and list comprehensions (used to create new lists from other iterables). Python also has automatic memory management eliminating the need to manually allocate and free memory in the code.

ANACONDA

Anaconda Cloud is a package management service by Anaconda. Cloud makes it easy to find, access, store and share public notebooks, environments, and conda and PyPI packages. Cloud also makes it easy to stay current with updates made to the packages and environments you are using. Cloud hosts hundreds of useful Python packages, notebooks, projects and environments for a wide variety of applications.

You do not need to log in, or even to have a Cloud account, to search for public packages, download and install them authentication, tokens, upload, download, remove and search. Connect to and manage your Anaconda Cloud account. Upload packages you have created. Generate access tokens to allow access to private packages.

For developers, Cloud is designed to make software development, release and maintenance easy by providing broad package management support. Cloud allows for free public package hosting, as well as package channels, providing a flexible and scalable service for groups and organizations of all sizes.

APPLICATIONS PROVIDED IN ANACONDA DISTRIBUTION

The Anaconda distribution comes with the following applications along with Anaconda Navigator.

- JupyterLab
- Jupyter Notebook
- Qt Console
- Spyder
- Glueviz
- Orange3
- Visual Studio Code

JupyterLab: This is an extensible working environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.

Jupyter Notebook: This is a web-based, interactive computing notebook environment. It is possible to edit and run human-readable docs while describing the data analysis.

Qt Console: It is the PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips and more.

Spyder: Spyder is a scientific Python Development Environment. It is a powerful Python IDE with advanced editing, interactive testing, debugging and introspection features.

VS Code: It is a streamlined code editor with support for development operations like debugging, task running and version control.

Glueviz: This is used for multidimensional data visualization across files. It explores relationships within and among related datasets.

Orange 3: It is a component-based data mining framework. This can be used for data visualization and data analysis. The workflows in Orange 3 are very interactive and provide a large toolbox.

3.4.2 HARDWARE SPECIFICATIONS

- Processor Type : Intel Core i5
- Speed : 3.40 GHz
- RAM : 4 GB
- Hard Disk : 500 GB
- Keyboard : 101/102 Standard Keys
- Mouse : Optical Mouse

3.4.3 SOFTWARE SPECIFICATIONS

- Operating System : Windows 10 / Windows 11
- Front End : Streamlit
- Coding Language : Python

CHAPTER – 4

SYSTEM DESIGN

4.1 SYSTEM DESIGN

4.1.1 INPUT DESIGN

Input design is the process of converting the user-originated input to a computer based format. The design decision for handling input specify how data are accepted for computer processing. Input design is a part of overall system design that needs careful attention.

The collection of input data is considered to be the most expensive part of the system design. Since the inputs have to be planned in such a way so as to get the relevant information, extreme care is taken to obtain the pertinent information. If the data going into the system is incorrect then the processing and outputs will magnify these errors. The goal of designing input data is to make data entry as easy, logical and free from errors as possible.

The following are the objectives of input design:

- To produce a cost effective method of input.
- To make the input forms understandable to the end users.
- To ensure the validation of data inputs.

The nature of input data is determined partially during logical system design. However, the nature of inputs is made more explicit during the physical design. The impact of inputs on the system is also determined. Effort has been made to ensure that input data remains accurate from the stage at which it is recorded and documented to the stage at which it is accepted by the computer. Validation procedures are also present to detect errors in data input, which is beyond control procedures. Validation procedures are designed to check each record, data item or field against certain criteria.

4.1.2 OUTPUT DESIGN

The output is designed in such a way that it is attractive, convenient and informative. Forms are designed in python with various features, which make the console output more pleasing. As the outputs are the most important sources of information to the users, better design should improve the system's relationships with us and also will help in decision-making. Form design elaborates the way output is presented and the layout available for capturing information.

In this proposed work, the output is designed in the form reports. The system is designed to generate various user friendly reports to help the business process. Figure 4.1 represent the different reports supported and Figure 4.2 represent the activity diagram :

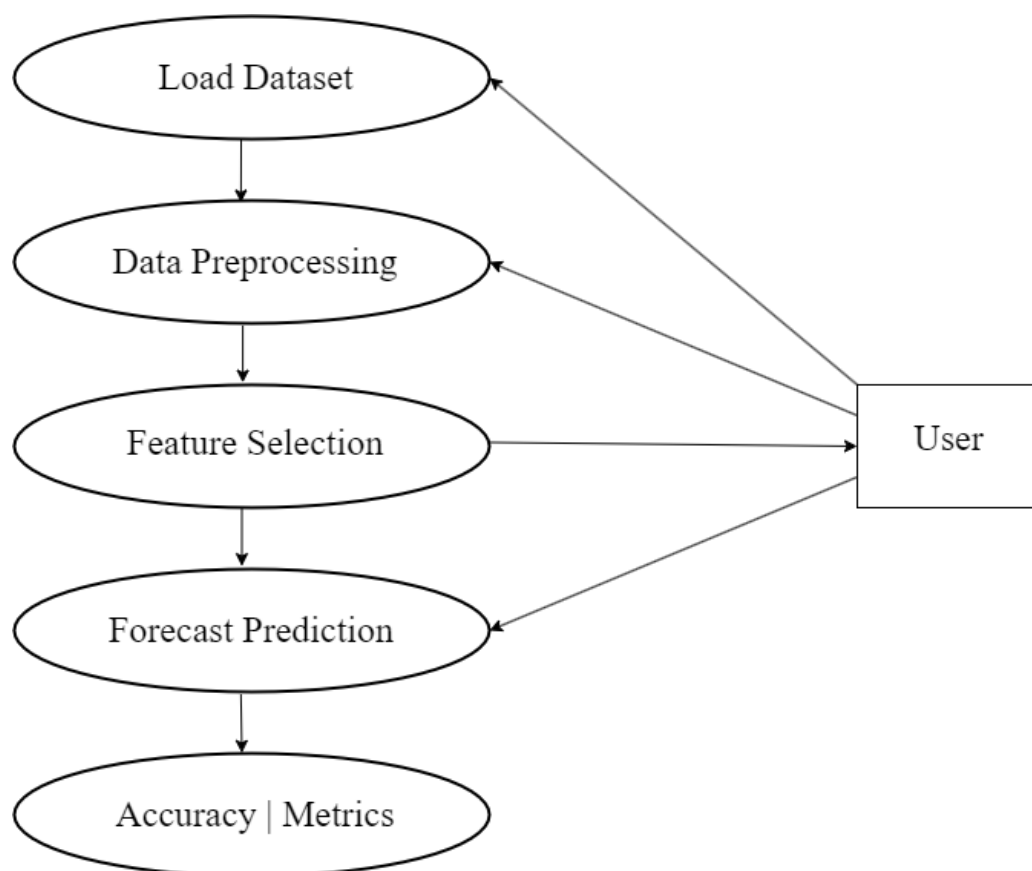


Fig 4.1 : USE CASE DIGAGRAM

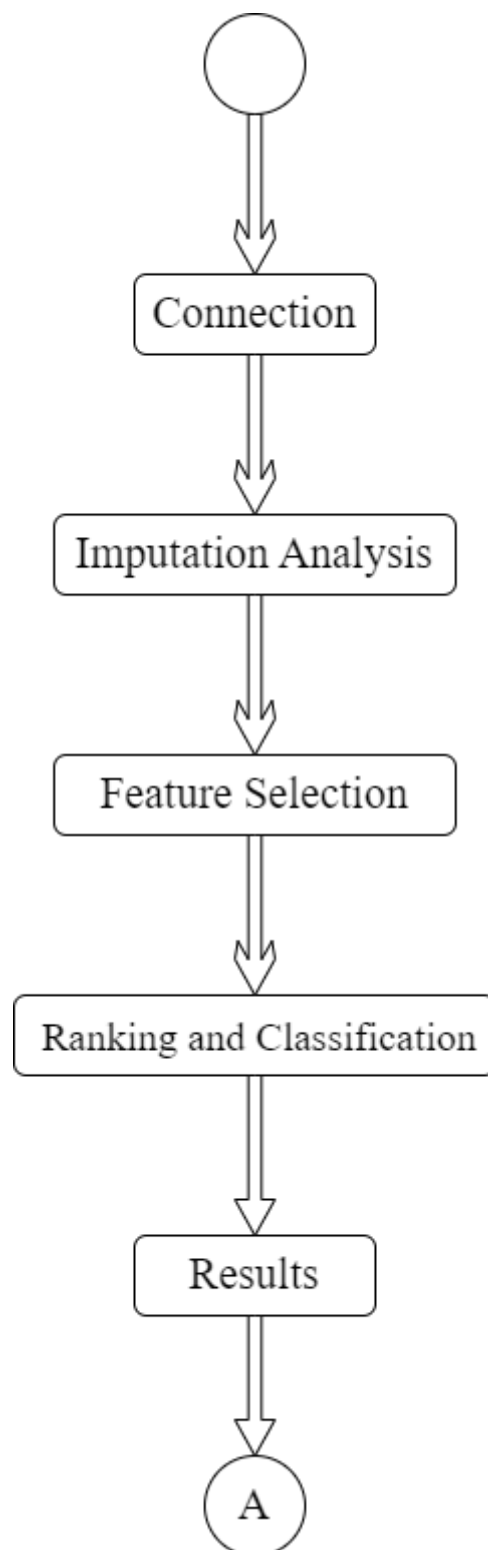


Fig 4.2 : ACTIVITY DIAGRAM

4.2 DESIGN CONSTRAINTS AND STANDARDS

The application considers two constraints namely, economic, ethical and social. Economic constraint is satisfied by providing plausible success rate and survey communications. This IEEE standard identifies and provides definitions for computed aided designs, character inference, computer assisted instruction, writer inference tasks, language processing, data processing and analysis among others. The system analyses dataset containing the day-wise actual past data and makes predictions for upcoming days using machine learning algorithms. Overall, we conclude that model predictions according to the current scenario are correct which may be helpful to understand the upcoming situation. The standard typically applies to automation tasks and learning techniques. These modules are implemented in the application subject to INSPEC Accession Number: 19660082.DOL: 10.1109/ACCESS 2020.2997311 IEEE Access

4.3 DESIGN ALTERNATIVES

The challenge of the stock price forecast is the most crucial component for companies and equity traders to predict future revenues. Predicting the market is challenging because the future is inherently unpredictable. Short-term traders are typically better served by waiting for confirmation that a reversal is at hand, rather than trying to predict a reversal will happen in the future. A successful and accurate prediction to the future stock prices ultimately results in profit maximisation. The challenges also includes that it is highly limited in its scope. Many predictors cannot be used, which is required to solve the stock price prediction problem. Various factors like demand and supply, economy, government policies, dividend declarations, political scenario also affect stock price prediction. To contribute to this aspect of information, numerous researchers are studying the different dimensions of applying machine learning algorithms to stabilise predictions.

4.4 MODULE DESCRIPTION

4.4.1 DETAILED DESIGN

Time series forecasting occurs when you make scientific predictions based on historical time stamped data. It involves building models through historical analysis and using them to make observations and drive future strategic decision-making. An important distinction in forecasting is that at the time of the work, the future outcome is completely unavailable and can only be estimated through careful analysis and evidence-based priors. Time series forecasting is the process of analysing time series data using statistics and modelling to make predictions and inform strategic decision-making. It's not always an exact prediction, and likelihood of forecasts can vary wildly—especially when dealing with the commonly fluctuating variables in time series data as well as factors outside our control. However, forecasting insight about which outcomes are more likely—or less likely—to occur than other potential outcomes. Often, the more comprehensive the data we have, the more accurate the forecasts can be.

The various Time series forecasting considerations are -

- Time horizons
- Dynamic and static states
- Data quality

4.5 DATA COLLECTION

The dataset includes stock information about a particular organization which is fetched using the Yahoo Finance open-source API available. The total size of the dataset varies according to the user who selects the number of years the user wants to analyse, train and predict.

4.6 DATA PRE-PROCESSING

The first step in creation of any Deep Learning Model is filtering the data which is to be given as input. Data pre-processing module involves encoding data into machine understandable format. It helps to clean, format and transform the raw data into efficient datasets, thereby making it ready for Machine Learning models. The steps involved are:

- Data and metadata are encoded from csv files into arrays using NumPy and pandas.
- The number of input variables are reduced which is known as dimensionality reduction.
- Data is scaled because scaling is a technique to make points in the data closer to each other.
- The various pre-processing techniques are applied to the dataset and passed to the models.
- The model is trained and predicts the future forecasted values for the particular stock,

4.7 TRAINING THE MODELS

- i. In the form of NumPy arrays the data and target data created is loaded.
- ii. Keras :
 - Shaping the data: Data in the form of NumPy array (data and target data) are converted to matrix form which is called shaping the data.
 - Weight is assigned to all data.
 - Different keras layers are assigned to the models according to their types.
 - Trainable and non-trainable parameters are given by the output layer.
- iii. Scikitlearn and tensorflow :
 - Using the tensorflow and scikitlearn python library data is split into training and Testing data.
- iv. Matplotlib :
 - To evaluate the model graphs are plotted which show training and validation loss.
- v. Finally, the model is evaluated and its prediction accuracy is calculated.

4.8 WEB APPLICATION

The backend is implemented in Python and the front end is implemented using Streamlit which uses python language. In python, the application links all the above-mentioned modules using their respective functions so that they are executed in a sequential manner:

- i. Data Pre-Processing
- ii. Index
- iii. Predict

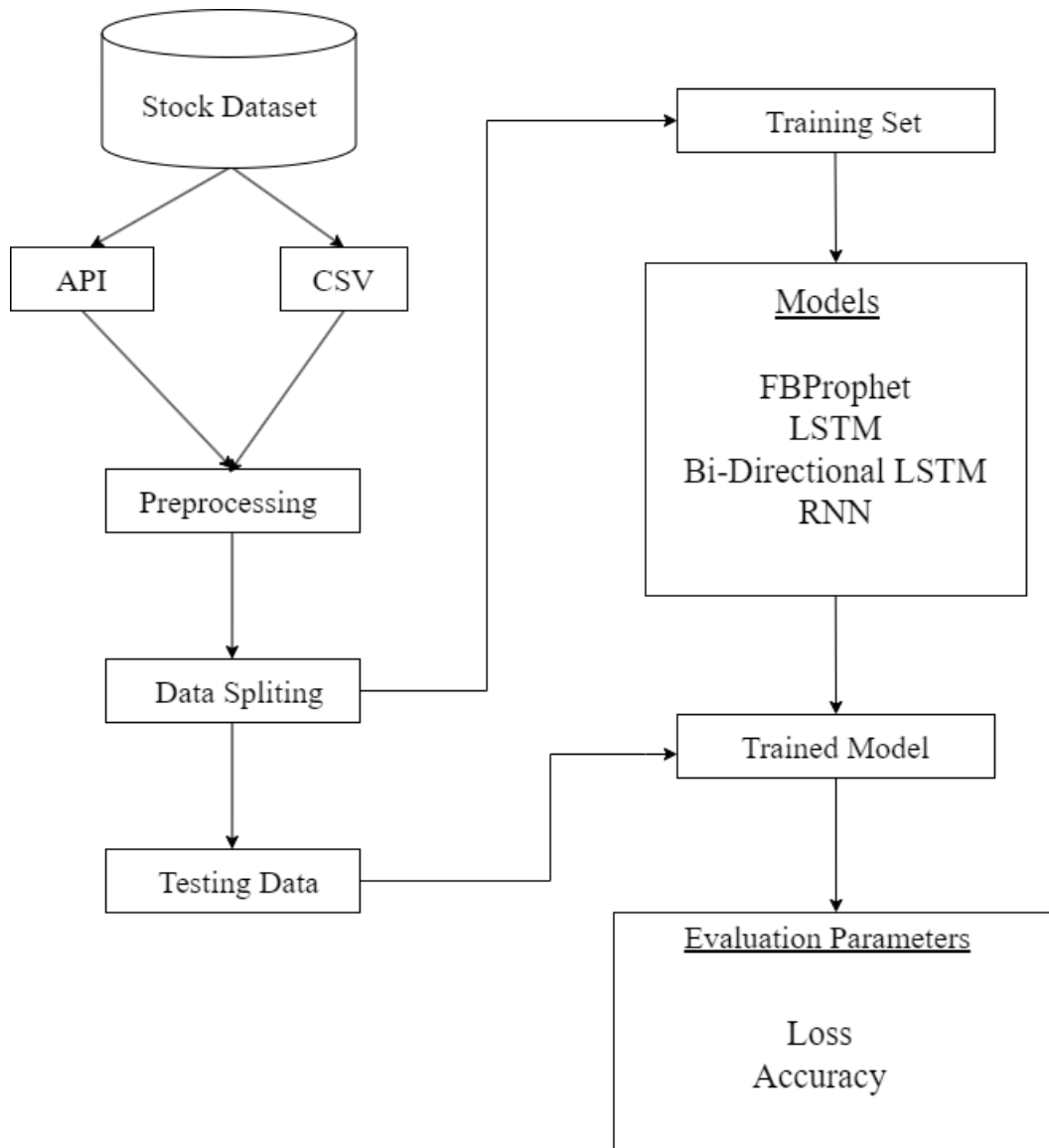


Fig 4.3 : PROPOSED WORKFLOW

Figure 4.3 represents the proposed workflow of the project. We pre-process the data and split the data into training and testing and train the model. The model is evaluated with the testing data and various evaluation parameters are obtained.

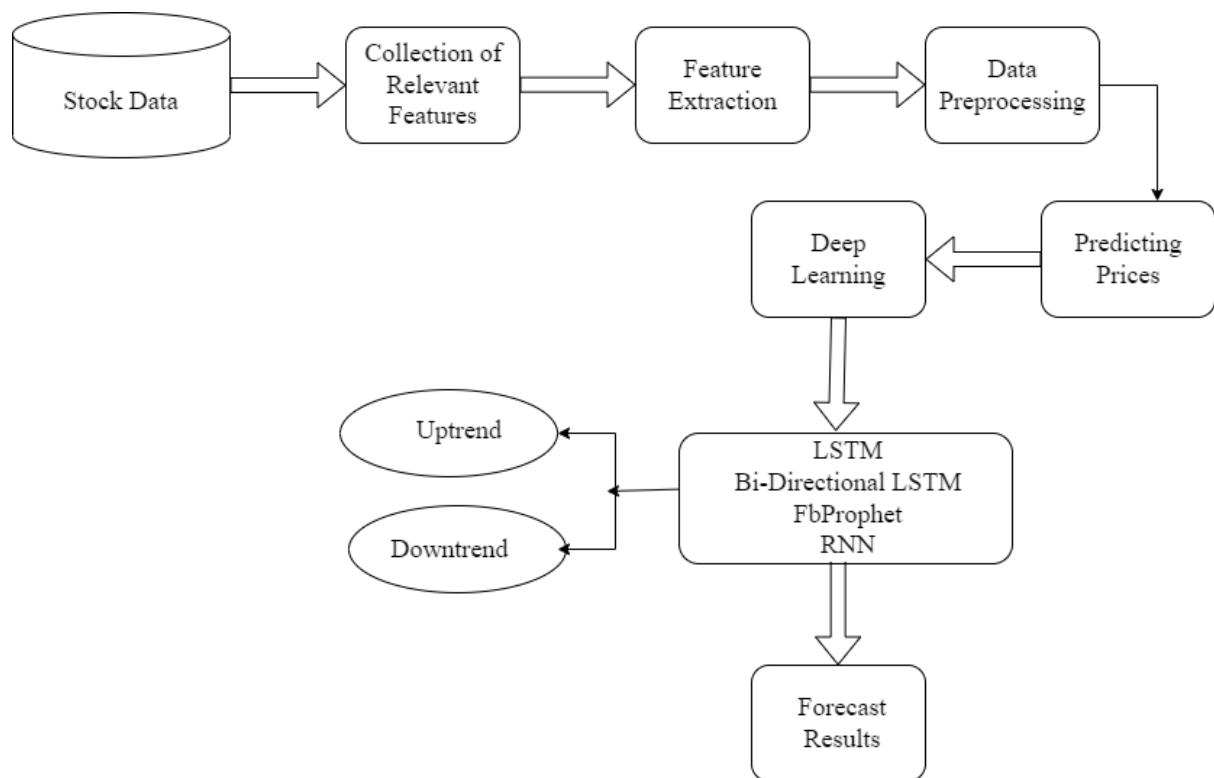


Fig 4.4 : OVERALL FLOW DIAGRAM

Figure 4.4 represents the overall flow diagram of the proposed model. We develop a system for the prediction and forecasting of stock prices using various Deep Learning and Forecasting Algorithms. The dataset used for the study contains the price and other necessary information about the particular stock. The visualisations in the graphs indicate the future trends and prices of the particular stock forecasted from the trained model.

CHAPTER – 5

IMPLEMENTATION

5.1 FRONT END CODE

```
import streamlit as st

from pandas_datareader import data as pdr

import yfinance as yfin

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import plotly.graph_objects as go

from plotly.subplots import make_subplots

from prophet import Prophet

from prophet.plot import plot_plotly

import datetime

from sklearn.preprocessing import MinMaxScaler

from keras.models import load_model

yfin.pdr_override()

with st.sidebar:

    choice = st.selectbox("Select the page",("Home","Analyse","Predict","FBPROPHET"))

if choice == "Home":

    st.header("Stock Price Prediction and Forecasting Using Machine Learning")

    st.markdown("---")
```

```
st.write("- Stock Price Prediction using machine learning helps you discover the future value  
of company stock and other financial assets traded on an exchange.")
```

```
st.write("- The entire idea of predicting stock prices is to gain significant profits. Predicting  
how the stock market will perform is a hard task to do.")
```

```
st.write("- There are other factors involved in the prediction, such as physical and  
psychological factors, rational and irrational behavior, and so on.")
```

```
st.write("- All these factors combine to make share prices dynamic and volatile.")
```

```
st.write("- This makes it very difficult to predict stock prices with high accuracy.")
```

5.2 BACK END CODE

```
elif choice == "Analyse":
```

```
    st.title("Stock Chart")
```

```
    user_input = st.text_input("Enter Stock Ticker", "AAPL")
```

```
    start = st.date_input("Enter Start Date", datetime.date(2023, 1, 1))
```

```
    end = st.date_input("Enter End Date")
```

```
    df = pdr.get_data_yahoo(user_input, start, end)
```

```
    df_time = df.reset_index()
```

```
    data = pd.DataFrame(df_time)
```

```
    submit = st.button("Submit")
```

```
    if submit:
```

```
        st.subheader("Stock Summary")
```

```
        st.table(data.describe())
```

```
        fig = make_subplots(rows=2, cols=1, shared_xaxes=True)
```

```
        fig.add_trace(go.Candlestick(x=data['Date'], open=data['Open'], high=data['High'],
```

```
                                low=data['Low'], close=data['Close'], name='AAPL'), row=1, col=1)
```

```

fig.add_trace(go.Bar(x=data["Date"],y=data["Volume"],name="Volume"),row=2,col=1)

fig.layout.update(title_text=f'Analysing Price and Volume of {user_input}',
xaxis_rangeslider_visible=True)

fig.update_layout(autosize=False, width=800, height=800)

st.plotly_chart(fig)

elif choice == "FBPROPHET":

    st.title("Time Series Forecasting using FBProphet")

    start = st.date_input("Enter Start Date", datetime.date(2023, 1, 1))

    end = st.date_input("Enter End Date")

    user_input = st.text_input("Enter Stock Ticker", "AAPL")

    years = st.slider("Number of Future Years to Forecast", 1, 5)

    period = years * 365

    yfin.pdr_override()

    df = pdr.get_data_yahoo(user_input, start, end)

    df_time = df.reset_index()

    data = pd.DataFrame(df_time)

    DFTrain = data[['Date', 'Close']]

    DFTrain = DFTrain.rename(columns = {"Date": "ds", "Close": "y"})

    submit = st.button("Submit")

    if submit :

        model = Prophet()

        model.fit(DFTrain)

        future = model.make_future_dataframe(periods = period)

        forecast = model.predict(future)

```

```

st.subheader('Forecasted Prices')

forecastFig = model.plot(forecast)

ax = forecastFig.gca()

ax.set_xlabel("Time", size=10)

ax.set_ylabel("Stock Price ($)", size=10)

ax.set_facecolor((0.08, 0.08, 0.1))

forecastFig.patch.set_facecolor((0.13, 0.18, 0.25))

ax.xaxis.label.set_color('white')

ax.tick_params(axis='x', colors='white')

ax.yaxis.label.set_color('white')

ax.tick_params(axis='y', colors='white')

st.pyplot(forecastFig)

elif choice == "Predict":

    st.title("Stock Trend Forecasting")

    choice = st.selectbox("Select the algorithm", ("LSTM", "BI-LSTM", "RNN"))

    model = load_model(f'{choice}.h5')

    start = "2013-01-01"

    end = "2023-03-04"

    user_input = st.text_input("Enter Stock Ticker", "AAPL")

    df = pdr.get_data_yahoo(user_input, start, end)

    n_days = st.slider('Forecast Days', 0, 45, 15)

    submit = st.button("Submit")

    if submit :

        ds = df['Close']

```

```

normalizer = MinMaxScaler(feature_range=(0,1))

ds_scaled = normalizer.fit_transform(np.array(ds).reshape(-1,1))

train_size = int(len(ds_scaled)*0.70)

test_size = len(ds_scaled) - train_size

ds_train, ds_test = ds_scaled[0:train_size,:], ds_scaled[train_size:len(ds_scaled),:]

def create_ds(dataset,step):

    Xtrain, Ytrain = [], []

    for i in range(len(dataset)-step-1):

        a = dataset[i:(i+step), 0]

        Xtrain.append(a)

        Ytrain.append(dataset[i + step, 0])

    return np.array(Xtrain), np.array(Ytrain)

X_train, y_train = create_ds(ds_train,100)

X_test, y_test = create_ds(ds_test,100)

X_train = X_train.reshape(X_train.shape[0],X_train.shape[1] , 1)

X_test = X_test.reshape(X_test.shape[0],X_test.shape[1] , 1)

train_predict = model.predict(X_train)

test_predict = model.predict(X_test)

y_predicted = model.predict(X_test)

train_predict = normalizer.inverse_transform(train_predict)

test_predict = normalizer.inverse_transform(test_predict)

test = np.vstack((train_predict,test_predict))

st.subheader("Scaling the data")

fig2 = plt.figure(figsize = (12,6))

```

```

plt.plot(normalizer.inverse_transform(ds_scaled))

plt.plot(test)

st.pyplot(fig2)

st.subheader("Predictions vs Original")

fig2 = plt.figure(figsize = (12,6))

plt.plot(y_test, 'b', label = 'Original Price')

plt.plot(y_predicted, 'r', label = 'Predicted Price')

plt.xlabel('Time')

plt.ylabel('Price')

plt.legend()

st.pyplot(fig2)

fut_inp = ds_test[len(ds_test)-100:]

fut_inp = fut_inp.reshape(1,-1)

tmp_inp = list(fut_inp)

tmp_inp = tmp_inp[0].tolist()

lst_output=[]

n_steps=100

i=0

while(i < n_days):

    if(len(tmp_inp)>100):

        fut_inp = np.array(tmp_inp[1:])

        fut_inp=fut_inp.reshape(1,-1)

        fut_inp = fut_inp.reshape((1, n_steps, 1))

        yhat = model.predict(fut_inp, verbose=0)

```

```

tmp_inp.extend(yhat[0].tolist())

tmp_inp = tmp_inp[1:]

lst_output.extend(yhat.tolist())

i=i+1

else:

    fut_inp = fut_inp.reshape((1, n_steps,1))

    yhat = model.predict(fut_inp, verbose=0)

    tmp_inp.extend(yhat[0].tolist())

    lst_output.extend(yhat.tolist())

    i=i+1

fig2 = plt.figure(figsize = (12,6))

plot_new=np.arange(1,101)

plot_pred=np.arange(101,101+n_days)

plt.plot(plot_new, normalizer.inverse_transform(ds_scaled[len(ds_scaled)-100:]))

plt.plot(plot_pred, normalizer.inverse_transform(lst_output))

st.pyplot(fig2)

ds_new = ds_scaled.tolist()

ds_new.extend(lst_output)

final_graph = normalizer.inverse_transform(ds_new).tolist()

fig2 = plt.figure(figsize = (12,6))

plt.plot(final_graph,)

plt.ylabel("Price")

plt.xlabel("Time")

plt.title(f"Prediction of Next {n_days} days")

```

```

plt.axhline(y=final_graph[len(final_graph)-1], color = 'red', linestyle = ':', label = 'NEXT
{0} DAY : {1}'.format(n_days,round(float(*final_graph[len(final_graph)-1]),2)))

plt.legend()

st.pyplot(fig2)

ranger = []

for i in range(0,n_days):

    ranger.append(i)

price = final_graph[len(final_graph)-n_days:]

unpacked_list = [item for sublist in price for item in sublist]

df = pd.DataFrame({"Date":ranger, "Price":unpacked_list})

fig = go.Figure()

fig.add_trace(go.Scatter(x=df["Date"], y=df["Price"], name="forecast"))

fig.layout.update(title_text='Time Series data with Rangeslider',
xaxis_rangeslider_visible=True)

st.plotly_chart(fig)

unpacked_list = [item for sublist in final_graph for item in sublist]

x_date = []

for i in range(len(ds_new)):

    x_date.append(i)

df = pd.DataFrame({"Date":x_date, "Price":unpacked_list})

fig = go.Figure()

fig.add_trace(go.Scatter(x=df["Date"],y=df["Price"],name="forecast", marker=dict(b)))

fig.layout.update(title_text='Time Series data with Rangeslider',
xaxis_rangeslider_visible=True)

st.plotly_chart(fig)

```

5.3 RESULTS

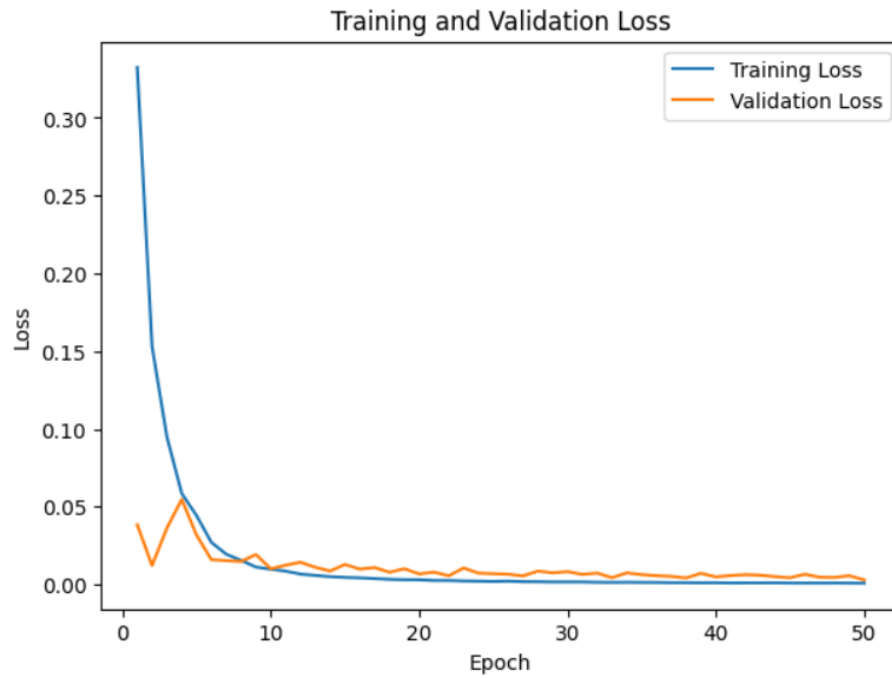


Fig 5.1 : LOSS GRAPH OF RNN MODEL



Fig 5.2 : LOSS GRAPH OF LSTM MODEL

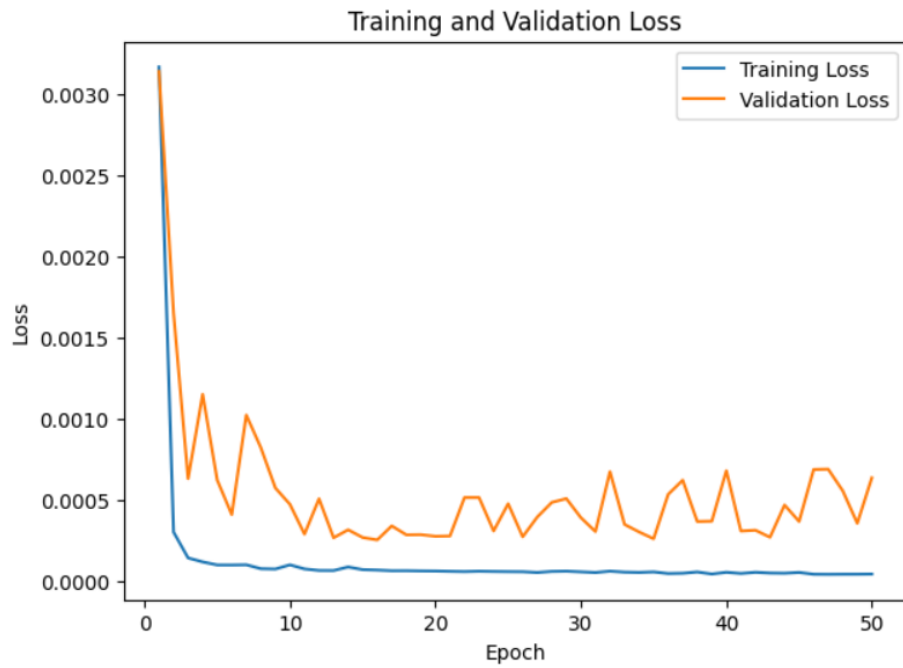


Fig 5.3 : LOSS GRAPH OF BI-LSTM MODEL

MODEL	EPOCHS	LOSS %
RNN	50	0.285
LSTM	50	0.063
BI-LSTM	50	0.088

Table 5.1 : MODEL METRICS

Table 5.1 contains the loss metrics of the models trained and evaluated.

CHAPTER - 6

TESTING

6.1 INTRODUCTION

Software testing is a critical element of software quality assurance and represents the ultimate review of specification, design and coding. In fact, testing is the one step in the software engineering process that could be viewed as destructive rather than constructive. A strategy for software testing integrates software test case design methods into a well-planned series of steps that result in the successful construction of software. Testing is the set of activities that can be planned in advance and conducted systematically. The underlying motivation of program testing is to affirm software quality with methods that can economically and effectively apply to both strategic to both large and small-scale systems.

6.2 STRATEGIC APPROACH TO SOFTWARE TESTING

The software engineering process can be viewed as a spiral. Initially system engineering defines the role of software and leads to software requirement analysis where the information domain, functions, behaviour, performance, constraints and validation criteria for software are established. Moving inward along the spiral, we come to design and finally to coding. To develop computer software we spiral in along streamlines that decrease the level of abstraction on each turn. A strategy for software testing may also be viewed in the context of the spiral. Unit testing begins at the vertex of the spiral and concentrates on each unit of the software as implemented in source code. Talking another turn on outward on the spiral we encounter validation testing where requirements established as part of software requirements analysis are validated against the software that has been constructed. Finally we arrive at system testing, where the software and other system elements are tested as a whole.

6.3 UNIT TESTING

Unit testing focuses verification effort on the smallest unit of software design, the module. The unit testing we have is white box oriented and some modules the steps are conducted in parallel.

6.3.1 WHITE BOX TESTING

To follow the concept of white box testing we have tested each form. We have created independently to verify that Data flow is correct and all conditions are exercised to check their validity.

- All loops are executed on their boundaries. This type of testing ensures that all independent paths have been exercised at least once
- All logical decisions have been exercised on their true and false sides
- All loops are executed at their boundaries and within their operational bounds
- All internal data structures have been exercised to assure their validity paths.

6.3.2 CONDITIONAL TESTING

In this part of the testing each of the conditions were tested to both true and false aspects. And all the resulting paths were tested so that each path that may generate on a particular condition is traced to uncover any possible errors.

This type of testing selects the path of the program according to the location of definition and use of variables. This kind of testing was used only when some local variable were declared. The definition-use chain method was used in this type of testing.

6.3.3 DATA FLOW TESTING

This type of testing selects the path of the program according to the location of definition and use of variables. This kind of testing was used only when some local variable were declared. The definition-use chain method was used in this type of testing.

6.3.4 LOOP TESTING

In this type of testing all the loops are tested to all the limits possible. The following exercise was adopted for all loops:

- All the loops were tested at their limits, just above them and just below them. All the loops were skipped at least once.
- For nested loops test the inner most loop first and then work outwards. For concatenated loops the values of dependent loops were set with the help of connected loop.
- Unstructured loops were resolved into nested loops or concatenated loops and tested as above.
- Each unit has been separately tested by the development team itself and all the input have been validated.

6.4 TEST CASE

A test case, in software engineering, is a set of conditions or variables under which a tester will determine whether an application, software system or one of its features is working as it was originally established for it to do. The mechanism for determining whether a software program or system has passed or failed such a test is known as a test oracle. In some settings, an oracle could be a requirement or use case, while in others it could be a heuristic. It may take many test cases to determine that a software program or system is considered sufficiently scrutinized to be released. Test cases are often referred to as test scripts, particularly when written - when they are usually collected into test suites.

6.4.1 TEST CASE 1

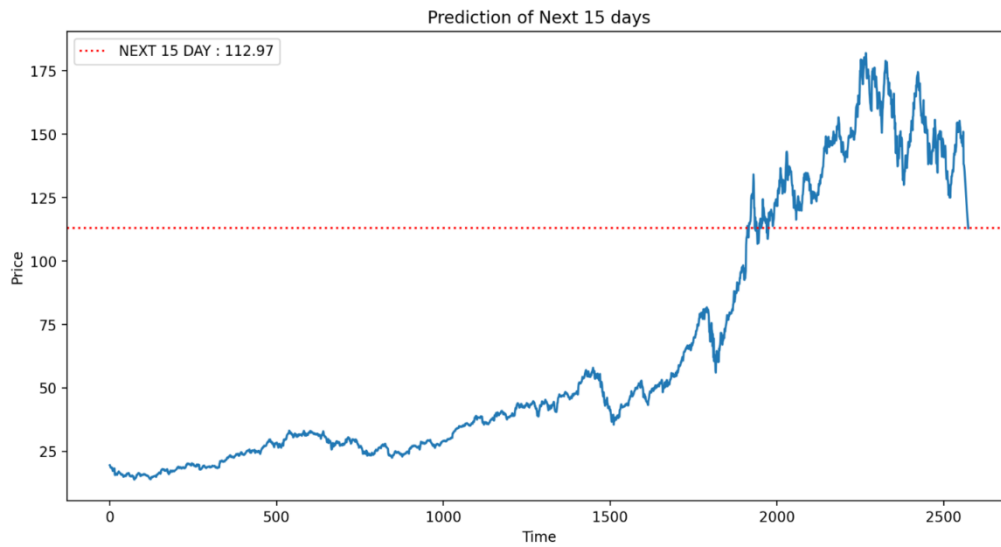


Fig 6.1 : TEST CASE 1

EXPECTED : Forecasted stock price for next 15 days

ACTUAL : Forecasted stock price for next 15 days

6.4.2 TEST CASE 2

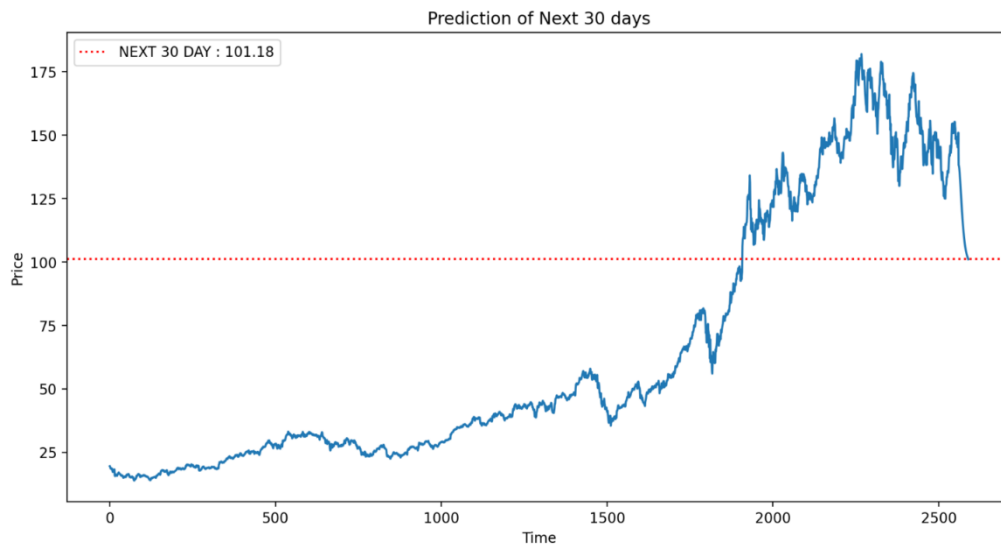


Fig 6.2 : TEST CASE 2

EXPECTED : Forecasted stock price for next 30 days

ACTUAL : Forecasted stock price for next 30 days

CHAPTER – 7

SCREENSHOTS

7.1 HOME PAGE

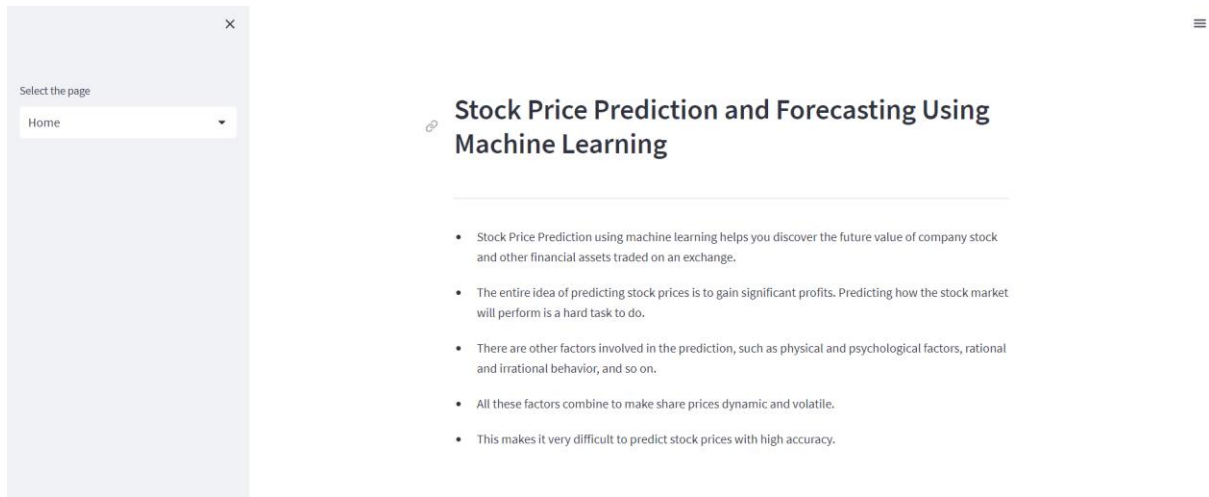


Fig 7.1 : HOME PAGE

Figure 7.1 represents the Home page which contains the basic details of Stock Prediction and Forecasting Project.

7.2 PREDICTIONS PAGE

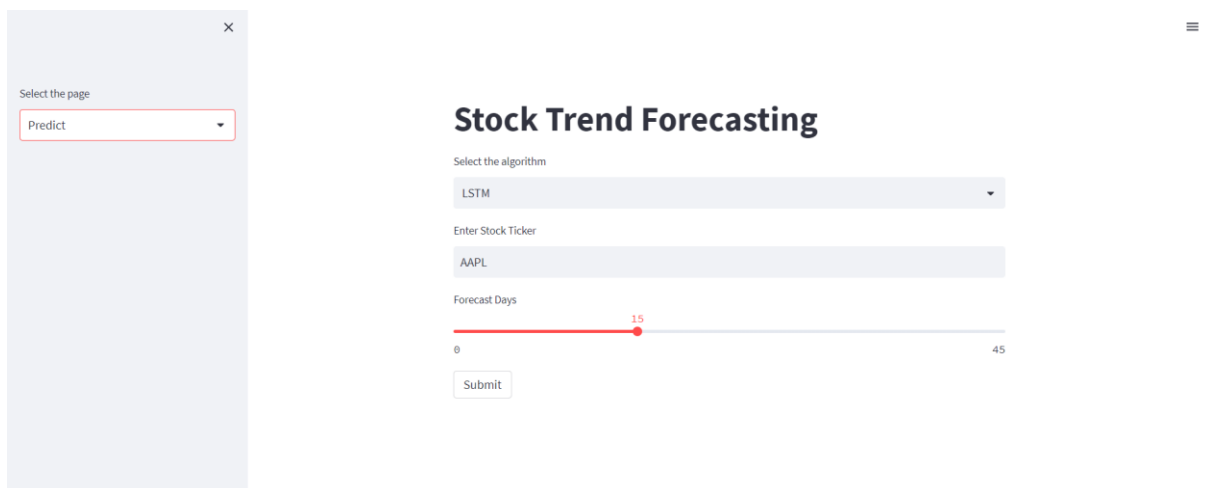


Fig 7.2 : PREDICTION PAGE

Figure 7.2 represents the prediction and future price forecasting page which is obtained as results from the model.

7.3 ANALYSE PAGE

×

Select the page

Analyse

Stock Chart

Enter Stock Ticker

AAPL

Enter Start Date

2023/01/01

Enter End Date

2023/04/09

Submit

Stock Summary

	Open	High	Low	Close	Adj Close	Volume
count	66.0000	66.0000	66.0000	66.0000	66.0000	66.0000
mean	147.9898	149.9782	146.7218	148.6120	148.5248	67,742,807.5758
std	10.2384	10.1267	10.4214	10.3652	10.4356	18,469,358.8050
min	126.0100	127.7700	124.1700	125.0200	124.8294	44,998,500.0000
25%	142.8150	144.2725	141.4650	143.2400	143.0216	55,611,025.0000
50%	150.1450	152.1700	148.2900	150.8450	150.6150	65,686,550.0000

Fig 7.3 : ANALYSE PAGE

Figure 7.3 represents the analyse page in which users can get statistical information about a stock.

7.4 FBPROPHET PAGE

×

Select the page

FBPROPHET

Time Series Forecasting using FBProphet

Enter Start Date

2023/01/01

Enter End Date

2023/04/09

Enter Stock Ticker

AAPL

Number of Future Years to Forecast

1

1

5

Submit

Fig 7.4 : FBPROPHET PAGE

Figure 7.4 represents the FbProphet page which contains the time-series forecasting algorithm model.

CHAPTER - 8

CONCLUSION AND FUTURE WORK

The project proposes a new approach for stock price prediction based on a different models like LSTM, Bidirectional-LSTM and RNN. Bidirectional - LSTM is a better approach also because more layers in the model reduce the generality of the results. In an effort to build a more efficient and reliable prediction system in the future, fundamental analysis and sentiment analysis will be included in the prediction process. By doing this, we will be able to gauge the general sentiment of the public regarding the company, which will allow us to tweak the prediction technique based on the sentiment and make even more accurate predictions in the future. Changing the way the missing data is handled, that is, replacing null values with running averages or median of a set range of data can also help increase training accuracy. An effort to reduce the training time as well as resource requirements of LSTM must be made. The problem of overfitting must also be taken care of, as LSTM can be sensitive to it.

Future scope of this project will involve adding more parameters and factors like the financial ratios, multiple instances, etc. The more the parameters are taken into account more will be the accuracy. The algorithms can also be applied for analyzing the contents of public comments and thus determine patterns/relationships between the customer and the corporate employee. The use of traditional algorithms and data mining techniques can also help predict the corporation performance structure as a whole. In the future, we plan to integrate neural network with some other techniques such as genetic algorithm or fuzzy logic. Genetic algorithm can be used to identify optimal network architecture and training parameters. Fuzzy logic provides the ability to account for some uncertainty produced by the neural network predictions. Their uses in conjunction with neural network could provide an improvement for stock market prediction.

REFERENCES

- [1] Novel Optimization Approach For Stock Price Forecasting Using Multi-Layered Sequential LSTM. *Applied Soft Computing* 134 (2023) 109830
- [2] Multi-scale local cues and hierarchical attention-based LSTM for stock price trend prediction. *Neurocomputing* 505 (2022) 92–100
- [3] Real-time forecasting of time series in financial markets using sequentially trained dual-LSTMs. *Expert Systems With Applications* 223 (2023) 119879
- [4] Forecasting Turning Points In Stock Price By Applying A Novel Hybrid CNN-LSTM-RESNET Model Fed By 2d Segmented Images. *Engineering Applications of Artificial Intelligence* 116 (2022) 105464
- [4] N.I. Sapankevych, R. Sankar, Time series prediction using support vector machines: a survey, *IEEE Comput. Intell. Mag.* 4 (2) (2009) 24–38.
- [5] P. Mondal, L. Shit, S. Goswami, Study of effectiveness of time series modeling (ARIMA) in forecasting stock prices, *Int. J. Comput. Sci. Eng. Appl.* 4 (2) (2014) 13.
- [6] Y. Chen, Z. Wei, X. Huang, Incorporating corporation relationship via graph convolutional neural networks for stock price prediction, in: *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*, 2018, pp. 1655–1658.
- [7] Y.T. Chen, W.N. Lai, E.W. Sun, Jump detection and noise separation by a singular wavelet method for predictive analytics of high-frequency data, *Comput. Econ.* 54 (2) (2019) 809–844.
- [8] N. Dalal, B. Triggs, Histograms of oriented gradients for human detection, in: *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05)* 1, 2005, pp. 886–893.
- [9] G.J. Deboeck, Trading on the edge: neural, genetic, and fuzzy systems for chaotic financial markets 39 (1994).
- [10] F. Feng, H. Chen, X. He, J. Ding, M. Sun, T.S. Chua, Enhancing stock movement prediction with adversarial training, *IJCAI* (2019)
- [11] K. Chen, Y. Zhou, F. Dai, A LSTM-based method for stock returns prediction: A case study of China stock market, in: *2015 IEEE International Conference on Big Data, big data, IEEE*, 2015, pp. 2823–2824.
- [12] F. Jiang, G. Tang, G. Zhou, Firm characteristics and Chinese stocks, *J. Manage. Sci. Eng.* 3 (4) (2018) 259–283.
- [13] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, *J. Mach. Learn. Res.* 13 (2) (2012).
- [14] A. Yadav, C.K. Jha, A. Sharan, Optimizing LSTM for time series prediction in Indian stock market, *Procedia Comput. Sci.* 167 (2020) 2091–2100.

- [15] J. Zhao, L. Itti, Classifying time series using local descriptors with hybrid sampling, *IEEE Trans. Knowl. Data Eng.* 28 (3) (2015) 623–637.
- [16] J. Zhao, L. Itti, shapedtw: Shape dynamic time warping, *Pattern Recogn.* 74 (2018) 171–184.
- [17] Y. Zhao, Y. Shen, Y. Zhu, J. Yao, Forecasting wavelet transformed time series with attentive neural networks, in: 2018 IEEE International Conference on Data Mining (ICDM), 2018, pp. 1452–1457.
- [18] X. Zhong, D. Enke, Predicting the daily return direction of the stock market using hybrid machine learning algorithms, *Financial Innov.* 5 (1) (2019) 4.
- [19] Y. Zhu, G. Zhou, Technical analysis: An asset allocation perspective on the use of moving averages, *J. Financ. Econ.* 92 (3) (2009) 519–544
- [20] K.j. Kim, Financial time series forecasting using support vector machines, *Neurocomputing* 55(1-2) (2003) 307–319

APPENDICES

APPENDIX - I

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from pandas_datareader import data as pdr

import yfinance as yfin

from sklearn.preprocessing import MinMaxScaler

import keras

from keras.models import Sequential

from keras.layers import Dense, LSTM, Bidirectional, SimpleRNN, Dropout

yfin.pdr_override()

data = pdr.get_data_yahoo('AAPL', start='2010-01-01', end='2020-01-01')

data.head()

ds = data["Open"].values

plt.plot(ds)


normalizer = MinMaxScaler(feature_range=(0,1))

ds_scaled = normalizer.fit_transform(np.array(ds).reshape(-1,1))

train_size = int(len(ds_scaled)*0.70)

test_size = len(ds_scaled) - train_size

ds_train, ds_test = ds_scaled[0:train_size,:], ds_scaled[train_size:len(ds_scaled),:]
```

```

def create_ds(dataset,step):

    Xtrain, Ytrain = [], []

    for i in range(len(dataset)-step-1):

        a = dataset[i:(i+step), 0]

        Xtrain.append(a)

        Ytrain.append(dataset[i + step, 0])

    return np.array(Xtrain), np.array(Ytrain)

X_train, y_train = create_ds(ds_train, 100)

X_test, y_test = create_ds(ds_test, 100)

X_train = X_train.reshape(X_train.shape[0],X_train.shape[1] , 1)

X_test = X_test.reshape(X_test.shape[0],X_test.shape[1] , 1)

```

```

def plot_loss(epoch):

    plt.figure(figsize=(7,5))

    epochs_range = range(1, epoch+1)

    train_loss = history.history['loss']

    val_loss = history.history['val_loss']

    plt.plot(epochs_range, train_loss, label="Training Loss")

    plt.plot(epochs_range, val_loss, label="Validation Loss")

    plt.title("Training and Validation Loss")

    plt.xlabel("Epoch")

    plt.ylabel("Loss")

    plt.legend()

    plt.show()

```

```

## RNN Model

epoch = 50

rnn_model = Sequential()

rnn_model.add(SimpleRNN(units = 50, activation = "tanh", return_sequences = True,
input_shape = (X_train.shape[1],1)))

rnn_model.add(Dropout(0.2))

rnn_model.add(SimpleRNN(units = 50, activation = "tanh", return_sequences = True))

rnn_model.add(Dropout(0.2))

rnn_model.add(SimpleRNN(units = 50, activation = "tanh", return_sequences = True))

rnn_model.add(Dropout(0.2))

rnn_model.add(SimpleRNN(units = 50))

rnn_model.add(Dropout(0.2))

rnn_model.add(Dense(units = 1))

rnn_model.summary()

rnn_model.compile(optimizer = "adam", loss = "mean_squared_error")

history = rnn_model.fit(X_train, y_train, epochs = epoch, batch_size = 32,
validation_data=(X_test, y_test))

rnn_loss = rnn_model.evaluate(X_test, y_test)

plot_loss(epoch)

rnn_model.save("RNN_model.h5")

```

```

## LSTM Model

epoch = 50

lstm_model = Sequential()

lstm_model.add(LSTM(units=50,return_sequences=True,input_shape=(X_train.shape[1],1)))

lstm_model.add(LSTM(units=50,return_sequences=True))

lstm_model.add(LSTM(units=50))

lstm_model.add(Dense(units=1,activation='linear'))

lstm_model.summary()

lstm_model.compile(loss='mean_squared_error',optimizer='adam')

history = lstm_model.fit(X_train,y_train,validation_data=(X_test,y_test),epochs=epoch,

                        batch_size=64)

lstm_loss = lstm_model.evaluate(X_test,y_test)

plot_loss(epoch)

lstm_model.save("lstm_model.h5")

```

Bi-LSTM Model

```

epoch = 50

bi_lstm_model = Sequential()

bi_lstm_model.add(Bidirectional(LSTM(units=128, input_shape=(X_train.shape[1], 1),

                                return_sequences=True))))

bi_lstm_model.add(Bidirectional(LSTM(units=64)))

bi_lstm_model.add(Dense(units=1))

bi_lstm_model.compile(optimizer='adam', loss='mean_squared_error')

history = bi_lstm_model.fit(X_train, y_train, epochs=epoch, batch_size=32,

                            validation_data=(X_test, y_test))

```

```
bi_lstm_model.summary()
```

```
bi_lstm_loss = bi_lstm_model.evaluate(X_test, y_test)
```

```
plot_loss(epoch)
```

```
bi_lstm_model.save("bi-lstm_model.h5")
```

```
data = {'Name':['RNN', 'Bi-LSTM', 'LSTM'],
```

```
        'Loss %': [rnn_loss*100, bi_lstm_loss*100, lstm_loss*100]}
```

```
df = pd.DataFrame(data)
```

```
plt.bar(df["Name"], df["Loss %"])
```

```
model = keras.models.load_model("bi-lstm_model.h5")
```

```
train_predict = model.predict(X_train)
```

```
test_predict = model.predict(X_test)
```

```
plt.plot(normalizer.inverse_transform(ds_scaled), 'r')
```

```
plt.plot(train_predict, 'g')
```

```
plt.plot(test_predict, 'b')
```

```
train_predict = normalizer.inverse_transform(train_predict)
```

```
test_predict = normalizer.inverse_transform(test_predict)
```

```
test = np.vstack((train_predict, test_predict))
```

```
plt.plot(normalizer.inverse_transform(ds_scaled))
```

```
plt.plot(test)
```

```
fut_inp = ds_test[len(ds_test)-100:]
```

```
fut_inp = fut_inp.reshape(1, -1)
```

```

tmp_inp = list(fut_inp)

tmp_inp = tmp_inp[0].tolist()


lst_output=[]

n_steps=100

i=0

while(i<30):

    if(len(tmp_inp)>100):

        fut_inp = np.array(tmp_inp[1:])

        fut_inp=fut_inp.reshape(1,-1)

        fut_inp = fut_inp.reshape((1, n_steps, 1))

        yhat = model.predict(fut_inp, verbose=0)

        tmp_inp.extend(yhat[0].tolist())

        tmp_inp = tmp_inp[1:]

        lst_output.extend(yhat.tolist())

        i=i+1

    else:

        fut_inp = fut_inp.reshape((1, n_steps,1))

        yhat = model.predict(fut_inp, verbose=0)

        tmp_inp.extend(yhat[0].tolist())

        lst_output.extend(yhat.tolist())

        i=i+1

plot_new=np.arange(1,101)

plot_pred=np.arange(101,131)

```

```

plt.plot(plot_new, normalizer.inverse_transform(ds_scaled[len(ds_scaled)-100:]))

plt.plot(plot_pred, normalizer.inverse_transform(lst_output))

ds_new = ds_scaled.tolist()

ds_new.extend(lst_output)

final_graph = normalizer.inverse_transform(ds_new).tolist()

plt.plot(final_graph,)

plt.ylabel("Price")

plt.xlabel("Time")

plt.title("Prediction of next month open")

plt.axhline(y=final_graph[len(final_graph)-1], color = 'red', linestyle = ':', label = 'NEXT 30D:
{0}'.format(round(float(*final_graph[len(final_graph)-1]),2)))

plt.legend()

```

APPENDIX – II

```
data.shape
```

```
(2516, 6)
```

```
print(train_size)
print(test_size)
```

```
1761
755
```

```
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
(1660, 100)
(1660,)
(654, 100)
(654,)
```

RNN MODEL :

```
rnn_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====	=====	=====
simple_rnn (SimpleRNN)	(None, 100, 50)	2600
dropout (Dropout)	(None, 100, 50)	0
simple_rnn_1 (SimpleRNN)	(None, 100, 50)	5050
dropout_1 (Dropout)	(None, 100, 50)	0
simple_rnn_2 (SimpleRNN)	(None, 100, 50)	5050
dropout_2 (Dropout)	(None, 100, 50)	0
simple_rnn_3 (SimpleRNN)	(None, 50)	5050
dropout_3 (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

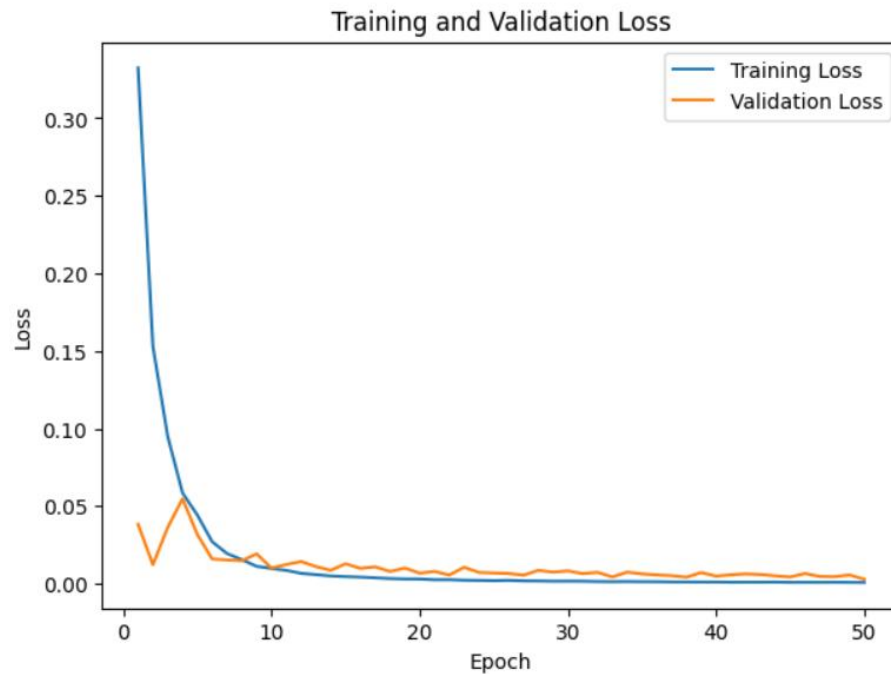
```
=====
Total params: 17,801
Trainable params: 17,801
Non-trainable params: 0
```

```
rnn_loss = rnn_model.evaluate(X_test, y_test)
```

```
21/21 [=====] - 0s 8ms/step - loss: 0.0028
```

```
0.002849999349564314
```

```
plot_loss(epoch)
```



LSTM MODEL :

```
lstm_model.summary()
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 100, 50)	10400
lstm_1 (LSTM)	(None, 100, 50)	20200
lstm_2 (LSTM)	(None, 50)	20200
dense_1 (Dense)	(None, 1)	51

```
=====
```

```
Total params: 50,851
```

```
Trainable params: 50,851
```

```
Non-trainable params: 0
```

```
lstm_loss = lstm_model.evaluate(X_test,y_test)
```

```
21/21 [=====] - 0s 18ms/step - loss: 8.8888e-04  
0.000888882961589843
```

```
plot_loss(epoch)
```



BI-LSTM MODEL :

```
bi_lstm_model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
bidirectional (Bidirectional)	(None, 100, 256)	133120
bidirectional_1 (Bidirectional)	(None, 128)	164352
dense_2 (Dense)	(None, 1)	129

Total params: 297,601

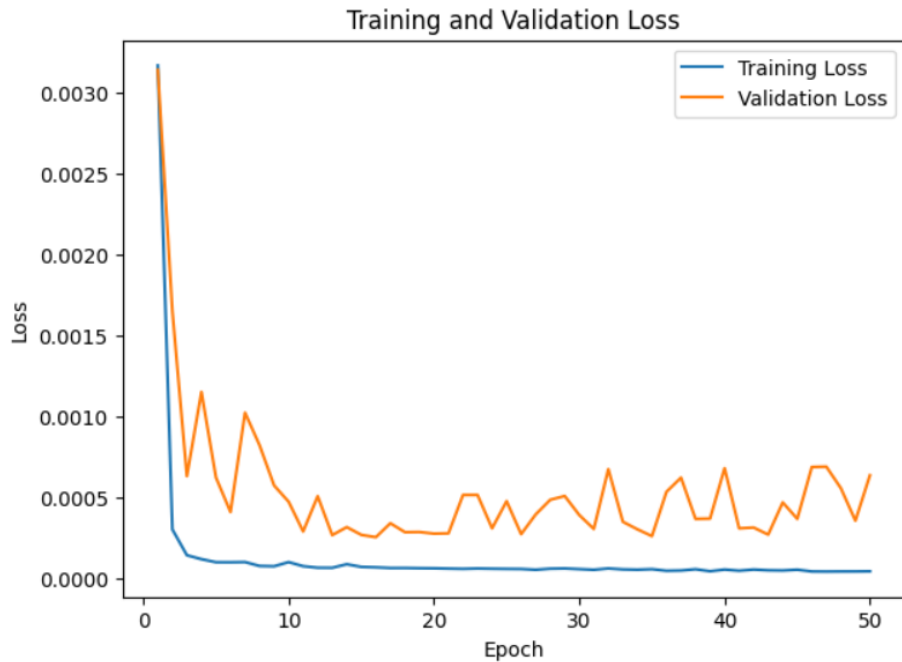
Trainable params: 297,601

Non-trainable params: 0

```
bi_lstm_loss = bi_lstm_model.evaluate(X_test, y_test)
```

```
21/21 [=====] - 2s 78ms/step - loss: 6.3960e-04  
0.0006395965465344489
```

```
plot_loss(epoch)
```



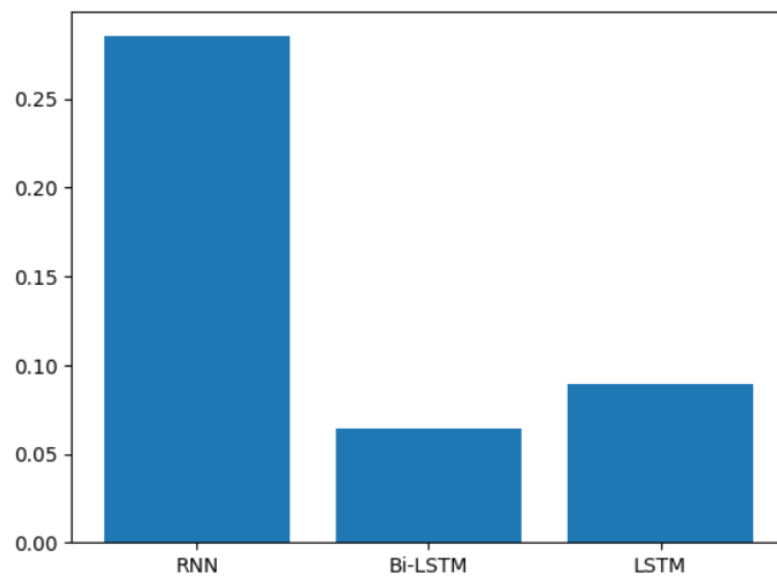
METRICS :

```
data = {'Name': ['RNN', 'Bi-LSTM', 'LSTM'],  
        'Loss %': [rnn_loss*100, bi_lstm_loss*100, lstm_loss*100]}  
  
df = pd.DataFrame(data)  
df
```

	Name	Loss %
0	RNN	0.285000
1	Bi-LSTM	0.063960
2	LSTM	0.088888

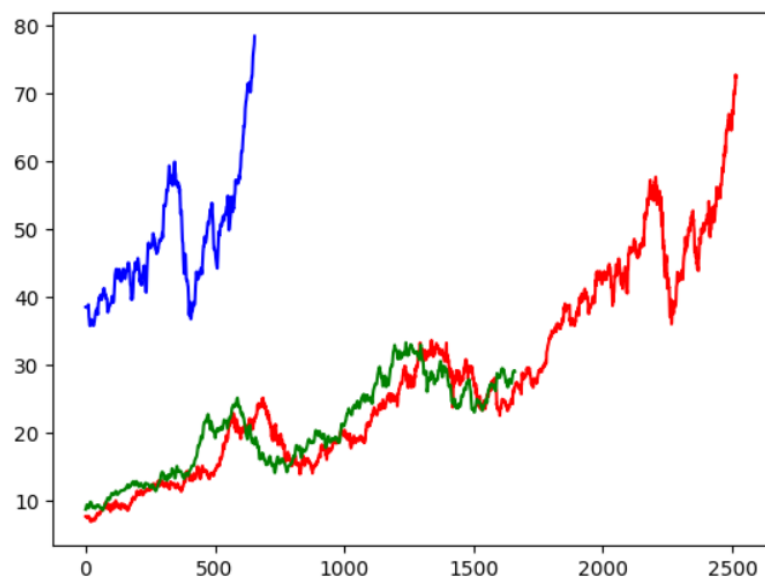
```
plt.bar(df["Name"], df["Loss %"])
```

<BarContainer object of 3 artists>



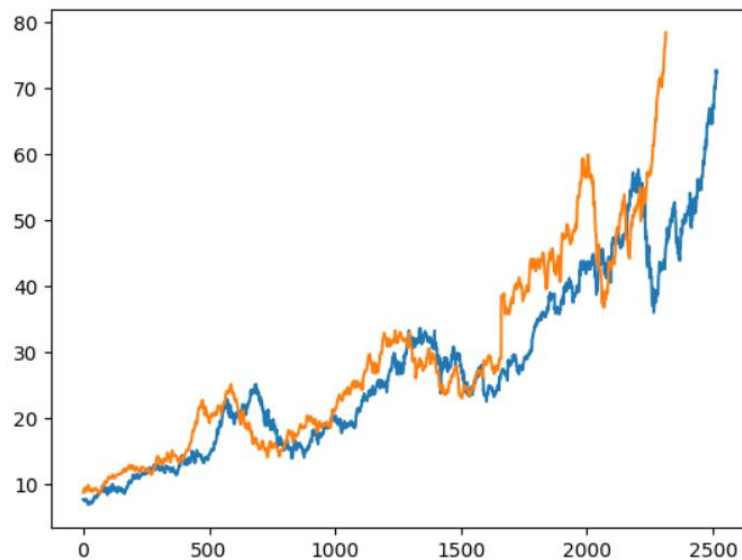
```
plt.plot(normalizer.inverse_transform(ds_scaled), 'r')  
plt.plot(train_predict, 'g')  
plt.plot(test_predict, 'b')
```

[<matplotlib.lines.Line2D at 0x18c93170ee0>]



```
plt.plot(normalizer.inverse_transform(ds_scaled))
plt.plot(test)
```

[<matplotlib.lines.Line2D at 0x18c8dd7b130>]



```
plt.plot(final_graph,)
plt.ylabel("Price")
plt.xlabel("Time")
plt.title("Prediction of next month open")
plt.axhline(y=final_graph[len(final_graph)-1], color = 'red', linestyle = ':',
            label = 'NEXT 30D: {}'.format(round(float(*final_graph[len(final_graph)-1]),
plt.legend()
```

<matplotlib.legend.Legend at 0x1dcd5359d60>

